



Upgrade scripts

An upgrade script is a Python file containing a function called `migrate()`, which the upgrade process invokes during the update of a module.

`migrate(cr, version)`

Parameters:

`cr` (`Cursor`) – current database cursor

`version` ([str](#)) – installed version of the module

Typically, this function executes one or multiple SQL queries and can also access Odoo's ORM, as well as the [Upgrade utils](#).

Writing upgrade scripts

Upgrade scripts follow a specific tree structure with a naming convention which determines when they are executed.

The structure of an upgrade script path is `$module/migrations/$version/pre,post,end-*.py`, where `$module` is the module for which the script will run, `$version` is the full version of the module (including Odoo's major version and the module's minor version) and `{pre|post|end}-*.py` is the file that needs to be executed. The file's name will determine the **phase** and order in which it is executed for that module and version.

Note

From Odoo 13 the top-level directory for the upgrade scripts can also be named `upgrades`. This naming is preferred since it has the correct meaning: *migrate* can be confused with *moving out of Odoo*. Thus `$module/upgrades/$version/` is also valid.

Note

Upgrade scripts are only executed when the module is being updated. Therefore, the module's minor version set in the `$version` directory needs to be higher than the module's installed version and equal or lower to the updated version of the module.

upgraded to version `2.0` on Odoo 17.

```
awesome_partner/  
|-- migrations/  
|   |-- 17.0.2.0/  
|   |   |-- pre-exclamation.py
```

Two upgrade scripts examples with the content of the `pre-exclamation.py`, file adding "!" at the end of partners' names:

```
import logging  
  
_logger = logging.getLogger(__name__)  
  
def migrate(cr, version):  
    cr.execute("UPDATE res_partner SET name = name || '!')"  
    _logger.info("Updated %s partners", cr.rowcount)
```

```
import logging  
from odoo.upgrade import util  
  
_logger = logging.getLogger(__name__)  
  
def migrate(cr, version):  
    env = util.env(cr)  
  
    partners = env["res.partner"].search([])  
    for partner in partners:  
        partner.name += "!"  
  
    _logger.info("Updated %s partners", len(partners))
```

Note that in the second example, the script takes advantage of the **Upgrade utils** to access the ORM. Check the documentation to find out more about this library.

1. The pre-phase, before the module is loaded.
2. The post-phase, after the module and its dependencies are loaded and updated.
3. The end-phase, after all modules have been loaded and updated for that version.

Upgrade scripts are grouped according to the first part of their filenames into the corresponding phase. Within each phase, the files are executed according to their lexical order.

Execution order of example scripts for one module in one version

1. *pre-10-do_something.py*
2. *pre-20-something_else.py*
3. *post-do_something.py*
4. *post-something.py*
5. *end-01-migrate.py*
6. *end-migrate.py*

Get Help

[Contact Support](#)[Ask the Odoo Community](#)